

Special Powers of Fast Tokenizers

In this section, we will look at the extra capabilities of tokenizers in Transformers.

So far, we have mainly used tokenizers for two tasks:

- splitting input text into tokens
- converting token IDs back into text

But tokenizers, especially those built on the Tokenizers library, can do much more.

To explain these extra features, this section shows how to manually reproduce the results of token-classification (NER) and question-answering pipelines.

What are slow tokenizers and fast tokenizers?

There are two types of tokenizers mentioned here:

Slow tokenizer

- written in Python inside the Transformers library

Fast tokenizer

- provided through the Tokenizers library
- written in Rust
- so it works much faster

Why is the fast tokenizer called “fast”?

The table below compares the speed of fast and slow tokenizers:

Fast tokenizer	Slow tokenizer
batched=True → 10.8s	4min 41s
batched=False → 59.2s	5min 3s

This means that when many texts are tokenized together, the fast tokenizer is much quicker.

Important point

When tokenizing only a single sentence, the fast tokenizer may not always be noticeably faster. Sometimes it may feel similar to the slow tokenizer, or even slightly slower.

But when tokenizing many texts together, the advantage of the fast tokenizer becomes very clear.

What is batch encoding?

The output of a tokenizer is not a normal Python dictionary.

It is actually a **BatchEncoding object**.

It behaves like a dictionary, so we can index into it like before. However, it also includes some extra methods, especially for fast tokenizers.

The most important feature of fast tokenizers

The most important power of a fast tokenizer is this:

It always keeps track of which part of the original text each final token came from.

This feature is called **offset mapping**.

Because of this, we can know:

- which token came from which word
- which character in the original text belongs to which token
- which token corresponds to which character range in the text

Example

```
from transformers import AutoTokenizer
```

```
tokenizer = AutoTokenizer.from_pretrained("bert-base-cased")  
example = "My name is Sylvain and I work at Hugging Face in Brooklyn."  
encoding = tokenizer(example)  
print(type(encoding))
```

The output will be:

```
<class 'transformers.tokenization_utils_base.BatchEncoding'>
```

This means the tokenizer returned a `BatchEncoding` object.

How to check whether it is a fast tokenizer

There are two ways to check.

1) From the tokenizer

```
tokenizer.is_fast
```

Output:

True

2) From the encoding

```
encoding.is_fast
```

Output:

True

What can we do with a fast tokenizer?

1) We can directly see the tokens

```
encoding.tokens()
```

Output:

```
[[CLS], 'My', 'name', 'is', 'S', '##yl', '##va', '##in', 'and', 'I', 'work', 'at', 'Hu', '##gging', 'Face', 'in', 'Brooklyn', '.', '[SEP]']
```

Here we can see that the word "Sylvain" has been split into several tokens:

- S
- ##yl

- `##va`
- `##in`

2) We can know which token came from which word

`encoding.word_ids()`

Output:

`[None, 0, 1, 2, 3, 3, 3, 3, 4, 5, 6, 7, 8, 8, 9, 10, 11, 12, None]`

This means:

- `[CLS]` and `[SEP]` are special tokens, so their value is `None`
- the rest show the index of the word each token came from

For example:

- `"Sylvain"` is word index `3` in the sentence
- so `S`, `##y1`, `##va`, and `##in` all have word ID `3`

Why is this useful?

This helps us understand:

- whether a token is at the beginning of a word
- whether two tokens are part of the same word

Earlier, we could guess this from BERT's `##` prefix, but that only works for BERT-like tokenizers.

The `word_ids()` method works for any fast tokenizer.

This is very useful in tasks like:

- NER
- POS tagging
- whole word masking

What counts as a “word” is not always the same

What a word is depends on the tokenizer.

Example:

I ' ll

Is it one word or two words?

That depends on the tokenizer's pre-tokenization rule.

Some tokenizers

- split only on spaces
- then "I ' ll" is treated as one word

Some tokenizers

- split on punctuation as well
- then "I ' ll" is treated as two words

Practice idea

If you tokenize "81s" using `bert-base-cased` and `roberta-base`, you may see differences.

Things to observe:

- how the token is split
- what word IDs are produced

The `sentence_ids()` method

There is another method:

- `sentence_ids()`

It tells us which sentence a token came from.

Although in paired input cases, similar information can also be obtained from `token_type_ids`.

Getting character positions from a word or token

With a fast tokenizer, we can also get the character range from the original text.

Useful methods:

- `word_to_chars()`
- `token_to_chars()`
- `char_to_word()`
- `char_to_token()`

Example

```
start, end = encoding.word_to_chars(3)
example[start:end]
```

Output:

Sylvain

Here, we found the text span corresponding to `word id = 3`.

In other words, we found the character range of `"Sylvain"` in the original sentence.

Why is this possible?

Because the fast tokenizer keeps track of the original text offsets for every token.

Using this offset list, we can understand the relationship between tokens and the original text.

Where is this useful in real work?

This feature can be used to manually reproduce pipeline results, such as:

- token classification
- named entity recognition
- question answering

Simple summary of the whole topic

Fast tokenizers have three major advantages:

1) Speed

They are much faster when many texts are tokenized together.

2) Token-to-word mapping

We can know which token came from which word.

3) Character span mapping

We can know which part of the original text a token came from.

If you want, I can also convert this into:

- very simple English notes, or
- line-by-line English translation.

Inside the token-classification pipeline

In Chapter 1, we first saw how to use **NER (Named Entity Recognition)**.

What is NER?

NER is a task where we identify which parts of a text refer to entities such as:

- Person
- Location
- Organization

Examples:

- `Sylvain` → Person
- `Hugging Face` → Organization
- `Brooklyn` → Location

What does `pipeline()` do?

In Chapter 2, we saw that `pipeline()` combines three steps to get predictions from raw text:

1) Tokenization

Split the text into tokens.

2) Passing through the model

Feed the tokenized input into the model.

3) Post-processing

Convert the model's raw output into a readable result.

The text says:

- the first two steps are the same as in other pipelines
- but the post-processing in a token-classification pipeline is a bit more complex

This is because it is not enough just to get predictions; we also need to interpret the tokens correctly as entities.

Getting the basic result with the pipeline

First, a token classification pipeline is created:

```
from transformers import pipeline
```

```
token_classifier = pipeline("token-classification")  
token_classifier("My name is Sylvain and I work at Hugging Face in Brooklyn.")
```

The default model here is:

`dbmdz/bert-large-cased-finetuned-conll03-english`

This model performs NER on sentences.

What does the output show?

Output:

```
[{'entity': 'I-PER', 'score': 0.9993828, 'index': 4, 'word': 'S', 'start': 11, 'end': 12},  
{ 'entity': 'I-PER', 'score': 0.99815476, 'index': 5, 'word': '##yl', 'start': 12, 'end': 14},  
{ 'entity': 'I-PER', 'score': 0.99590725, 'index': 6, 'word': '##va', 'start': 14, 'end': 16},  
{ 'entity': 'I-PER', 'score': 0.9992327, 'index': 7, 'word': '##in', 'start': 16, 'end': 18},  
{ 'entity': 'I-ORG', 'score': 0.97389334, 'index': 12, 'word': 'Hu', 'start': 33, 'end': 35},  
{ 'entity': 'I-ORG', 'score': 0.976115, 'index': 13, 'word': '##gging', 'start': 35, 'end': 40},  
{ 'entity': 'I-ORG', 'score': 0.98879766, 'index': 14, 'word': 'Face', 'start': 41, 'end': 45},  
{ 'entity': 'I-LOC', 'score': 0.99321055, 'index': 16, 'word': 'Brooklyn', 'start': 49, 'end': 57}]
```

What does this output mean?

The model made a separate prediction for each token.

For "Sylvain"

The word was split into several tokens:

- S
- ##yl
- ##va
- ##in

All of them were labeled I-PER.

Meaning:

- they are parts of a **Person** entity

For "Hugging Face"

The tokens are:

- Hu
- ##gging
- Face

They were labeled I-ORG.

Meaning:

- they are parts of an **Organization** entity

For "Brooklyn"

The token Brooklyn was labeled I-LOC.

Meaning:

- it is a **Location**

What do the output fields mean?

Each item contains several pieces of information:

entity

What kind of entity it is

- **I-PER** = part of a person entity
- **I-ORG** = part of an organization entity
- **I-LOC** = part of a location entity

score

How confident the model is

index

The token position

word

The token itself

start, end

The character positions in the original text

For example:

- **Sylvain** appears between positions 11 and 18 in the text

What is the problem?

Here, the output is at the **token level**.

That means:

- **Sylvain** is shown as 4 separate tokens instead of one full word
- **Hugging Face** is also shown as separate tokens

But usually we want the final result at the **word/entity level**.

Grouping the tokens together

For this, we use `aggregation_strategy="simple"` in the pipeline:

```
from transformers import pipeline
```

```
token_classifier = pipeline("token-classification", aggregation_strategy="simple")  
token_classifier("My name is Sylvain and I work at Hugging Face in Brooklyn.")
```

Output:

```
[{'entity_group': 'PER', 'score': 0.9981694, 'word': 'Sylvain', 'start': 11, 'end': 18},  
 {'entity_group': 'ORG', 'score': 0.97960204, 'word': 'Hugging Face', 'start': 33, 'end': 45},  
 {'entity_group': 'LOC', 'score': 0.99321055, 'word': 'Brooklyn', 'start': 49, 'end': 57}]
```

What changed now?

Now the tokens have been grouped together.

Before:

- S, ##yl, ##va, ##in

Now:

- Sylvain

Before:

- Hu, ##gging, Face

Now:

- Hugging Face

This is much more readable.

What is **aggregation_strategy**?

It decides how to compute the final score when multiple tokens form one entity.

1) "simple"

Take the mean of the scores of all tokens in the entity.

Example:

- the score of **Sylvain** = average of the scores of **S**, **##yl**, **##va**, and **##in**

Other available strategies

2) "first"

Use the score of the first token in the entity.

Example:

- the score of **Sylvain** would be the score of **S**

3) "max"

Use the maximum score among all tokens in the entity.

Example:

- the score of **Hugging Face** would be the highest score among its tokens

4) "average"

Average the scores of the words that make up the entity.

This is close to "simple", but calculated a little differently, especially when the entity has multiple words.

Difference between "simple" and "average"

This is important.

"simple"

It averages the scores of all tokens.

"average"

It first computes word-level scores, then averages the words.

Example:

Hugging Face

Here:

- `Hugging` is made of several tokens
- `Face` is one word

With the "`average`" strategy:

- it first computes the score of `Hugging`
- then takes the score of `Face`
- then averages those two word scores

From inputs to predictions

This section shows how the model produces predictions from raw text.

The main idea is:

- tokenize the text
- send the tokens to the model
- get probabilities and predictions from the model's output logits
- then understand which token belongs to which entity

Step 1: Load the tokenizer and model

```
from transformers import AutoTokenizer, AutoModelForTokenClassification
```

```
model_checkpoint = "dbmdz/bert-large-cased-finetuned-conll03-english"  
tokenizer = AutoTokenizer.from_pretrained(model_checkpoint)  
model = AutoModelForTokenClassification.from_pretrained(model_checkpoint)
```

```
example = "My name is Sylvain and I work at Hugging Face in Brooklyn."  
inputs = tokenizer(example, return_tensors="pt")  
outputs = model(**inputs)
```

What happened here?

- `AutoTokenizer` converted the text into tokens
- `AutoModelForTokenClassification` loaded the NER model
- `inputs = tokenizer(...)` converted the text into the model's input format
- `outputs = model(**inputs)` made the model produce predictions

Step 2: Understand the output shape

```
print(inputs["input_ids"].shape)
print(outputs.logits.shape)
```

Output:

```
torch.Size([1, 19])
torch.Size([1, 19, 9])
```

What does this mean?

inputs["input_ids"].shape = [1, 19]

This means:

- batch size = 1
- total tokens = 19

So there is 1 sentence, and it was split into 19 tokens.

outputs.logits.shape = [1, 19, 9]

This means:

- 1 sentence
- 19 tokens
- 9 label scores for each token

In other words, the model gives 9 possible class scores for each token.

Step 3: Get probabilities and predictions from logits

```
import torch
```

```
probabilities = torch.nn.functional.softmax(outputs.logits, dim=-1)[0].tolist()
predictions = outputs.logits.argmax(dim=-1)[0].tolist()
print(predictions)
```

Output:

```
[0, 0, 0, 0, 4, 4, 4, 4, 0, 0, 0, 0, 6, 6, 6, 0, 8, 0, 0]
```

What is happening here?

What are logits?

The model does not directly output the final label. It first gives raw scores, which are called **logits**.

What is softmax?

`softmax()` converts logits into probabilities.

That means:

- it tells us how likely each label is

What is argmax?

`argmax()` picks the label index with the highest score.

That means:

- it decides which class will be the final prediction

Step 4: Look at the label mapping

`model.config.id2label`

Output:

```
{
  0: 'O',
  1: 'B-MISC',
  2: 'I-MISC',
  3: 'B-PER',
  4: 'I-PER',
  5: 'B-ORG',
  6: 'I-ORG',
  7: 'B-LOC',
  8: 'I-LOC'
}
```

What do these labels mean?

O

This means the token is not part of any entity.

O = Outside

B-XXX

Beginning of an entity

I-XXX

Inside an entity

For example:

- B-PER = beginning of a Person entity
- I-PER = inside a Person entity
- B-ORG = beginning of an Organization entity
- I-ORG = part of an Organization entity
- B-LOC = beginning of a Location entity
- I-LOC = part of a Location entity

What would we expect in this example?

Suppose "Sylvain" is tokenized as:

- S
- ##yl
- ##va
- ##in

Normally, we might expect:

- S to be B-PER
- the remaining tokens to be I-PER

But the model gave I-PER for all of them.

Did the model make a mistake?

Not completely.

This is related to the **IOB format**.

IOB1 and IOB2

There are two formats for entity labeling:

IOB2

Here, **B-** is used whenever an entity begins.

Example:

- **S** → **B-PER**
- **##y1** → **I-PER**

IOB1

Here, **B-** is not always used at the beginning.

It is mainly used to separate two adjacent entities of the same type.

The dataset used to fine-tune this model follows the **IOB1 format**.

That is why the token **S** can also get the label **I-PER**.

So the model's output is actually correct according to the dataset convention.

Step 5: Keep only the useful tokens, excluding **0**

```
results = []
```

```
tokens = inputs.tokens()
```

```
for idx, pred in enumerate(predictions):
```

```
    label = model.config.id2label[pred]
```

```
    if label != "O":
```

```
        results.append(
```

```
            {"entity": label, "score": probabilities[idx][pred], "word": tokens[idx]}
```

```
        )
```

```
print(results)
```

The output will look like this:

```
[
  {'entity': 'I-PER', 'score': 0.9993828, 'word': 'S'},
  {'entity': 'I-PER', 'score': 0.99815476, 'word': '##yl'},
  {'entity': 'I-PER', 'score': 0.99590725, 'word': '##va'},
  {'entity': 'I-PER', 'score': 0.9992327, 'word': '##in'},
  {'entity': 'I-ORG', 'score': 0.97389334, 'word': 'Hu'},
  {'entity': 'I-ORG', 'score': 0.976115, 'word': '##gging'},
  {'entity': 'I-ORG', 'score': 0.98879766, 'word': 'Face'},
  {'entity': 'I-LOC', 'score': 0.99321055, 'word': 'Brooklyn'}
]
```

What happened here?

We:

- looked at all tokens
- kept only the ones whose label was not O
- stored the entity, score, and word for each one

This gives us the token-level NER result.

What is still missing?

At this point, each token has:

- a label
- a score
- a word

But we still do not have the **start** and **end** character positions in the original sentence.

The pipeline had provided that information as well.

To get that, we need **offset mapping**.

Step 6: Get offset mapping

```
inputs_with_offsets = tokenizer(example, return_offsets_mapping=True)
inputs_with_offsets["offset_mapping"]
```

Output:

[(0, 0), (0, 2), (3, 7), (8, 10), (11, 12), (12, 14), (14, 16), (16, 18), (19, 22), (23, 24), (25, 29), (30, 32),
(33, 35), (35, 40), (41, 45), (46, 48), (49, 57), (57, 58), (0, 0)]

What is offset mapping?

It gives the character range in the original text for each token, as tuples.

For example:

- (11, 12) means characters from position 11 to 12
- (12, 14) means characters from position 12 to 14

Why (0, 0)?

These are used for special tokens, such as:

- [CLS]
- [SEP]

Example

Suppose the token at index 5 is ##yl

Its offset is (12, 14)

example[12:14]

Output:

yl

Notice:

- the token was ##yl
- but the original text slice is yl

This means offset mapping correctly identifies the token's position in the original text.

Step 7: Add start and end to the final result

```
results = []
inputs_with_offsets = tokenizer(example, return_offsets_mapping=True)
tokens = inputs_with_offsets.tokens()
offsets = inputs_with_offsets["offset_mapping"]

for idx, pred in enumerate(predictions):
    label = model.config.id2label[pred]
    if label != "O":
        start, end = offsets[idx]
        results.append(
            {
                "entity": label,
                "score": probabilities[idx][pred],
                "word": tokens[idx],
                "start": start,
                "end": end,
            }
        )

print(results)
```

Output:

```
[
  {'entity': 'I-PER', 'score': 0.9993828, 'word': 'S', 'start': 11, 'end': 12},
  {'entity': 'I-PER', 'score': 0.99815476, 'word': '##yl', 'start': 12, 'end': 14},
  {'entity': 'I-PER', 'score': 0.99590725, 'word': '##va', 'start': 14, 'end': 16},
  {'entity': 'I-PER', 'score': 0.9992327, 'word': '##in', 'start': 16, 'end': 18},
  {'entity': 'I-ORG', 'score': 0.97389334, 'word': 'Hu', 'start': 33, 'end': 35},
  {'entity': 'I-ORG', 'score': 0.976115, 'word': '##gging', 'start': 35, 'end': 40},
  {'entity': 'I-ORG', 'score': 0.98879766, 'word': 'Face', 'start': 41, 'end': 45},
  {'entity': 'I-LOC', 'score': 0.99321055, 'word': 'Brooklyn', 'start': 49, 'end': 57}
]
```

What do we get now?

Now for each entity token we have:

- the label
- the score
- the token
- the start position in the original text
- the end position in the original text

This is the same as the first pipeline result.

The whole idea in simple words

What did we do first?

- tokenized the text
- passed it to the model
- got the logits

What did we do next?

- used softmax to convert logits into probabilities
- used argmax to get the final labels
- used `id2label` to get the label names
- removed the `O` tokens and kept the entity tokens
- used offset mapping to find the start and end positions

Most important points

1) The model predicts token by token

It does not predict word by word; it predicts token by token.

2) Output shape `[1, 19, 9]`

This means:

- 1 sentence
- 19 tokens
- 9 labels for each token

3) `O` means no entity

4) `B-` and `I-`

- B- = beginning
- I- = inside

5) Offset mapping is very important

Because it tells us where each token came from in the original sentence.

From inputs to predictions

This section shows how a model generates predictions step by step from an input sentence.

Step 1: Tokenize the input and pass it to the model

First, we load the tokenizer and the model:

```
from transformers import AutoTokenizer, AutoModelForTokenClassification

model_checkpoint = "dbmdz/bert-large-cased-finetuned-conll03-english"
tokenizer = AutoTokenizer.from_pretrained(model_checkpoint)
model = AutoModelForTokenClassification.from_pretrained(model_checkpoint)

example = "My name is Sylvain and I work at Hugging Face in Brooklyn."
inputs = tokenizer(example, return_tensors="pt")
outputs = model(**inputs)
```

What is happening here:

- `AutoTokenizer` splits the sentence into tokens
- `AutoModelForTokenClassification` predicts labels for those tokens
- `return_tensors="pt"` means the output will be returned as PyTorch tensors

Step 2: Understand the model output shape

```
print(inputs["input_ids"].shape)
print(outputs.logits.shape)
```

Output:

```
torch.Size([1, 19])  
torch.Size([1, 19, 9])
```

This means:

- batch size = 1
- total tokens = 19
- the model gives 9 label scores for each token

So the output shape of the model is:

1 x 19 x 9

Meaning:

- 1 sentence
- 19 tokens
- 9 possible classes for each token

Step 3: Get predictions from logits

The model does not directly output the final label. It first produces raw scores, called **logits**.

Then `softmax` is used to convert them into probabilities, and `argmax` is used to select the most likely label.

```
import torch
```

```
probabilities = torch.nn.functional.softmax(outputs.logits, dim=-1)[0].tolist()  
predictions = outputs.logits.argmax(dim=-1)[0].tolist()  
print(predictions)
```

Output:

```
[0, 0, 0, 0, 4, 4, 4, 4, 0, 0, 0, 0, 6, 6, 6, 0, 8, 0, 0]
```

Here:

- `softmax()` converts raw logits into probabilities
- `argmax()` takes the label index with the highest score

Step 4: Understand the meaning of the label indices

model.config.id2label

Output:

```
{
  0: 'O',
  1: 'B-MISC',
  2: 'I-MISC',
  3: 'B-PER',
  4: 'I-PER',
  5: 'B-ORG',
  6: 'I-ORG',
  7: 'B-LOC',
  8: 'I-LOC'
}
```

Here:

- **O** means the token does not belong to any entity
- **B-XXX** means the beginning of an entity
- **I-XXX** means the inside part of an entity

Examples:

- **B-PER** = beginning of a person entity
- **I-PER** = inside a person entity
- **B-ORG** = beginning of an organization entity
- **I-ORG** = part of an organization entity
- **B-LOC** = beginning of a location entity
- **I-LOC** = part of a location entity

Step 5: Why did the token **S** also get **I-PER**?

At first, we might think:

- **S** should be **B-PER**
- **##y1, ##va, ##in** should be **I-PER**

But the model gave **I-PER** even to **S**.

This is not completely wrong, because there are two entity-labeling formats:

IOB2

Here, **B-** is used whenever an entity starts.

IOB1

Here, **B-** is mainly used when two entities of the same type are next to each other and need to be separated.

The dataset used to fine-tune this model follows the **IOB1** format. That is why the token **S** can also receive the label **I-PER**.

Step 6: Extract all entity tokens except **O**

```
results = []
tokens = inputs.tokens()

for idx, pred in enumerate(predictions):
    label = model.config.id2label[pred]
    if label != "O":
        results.append(
            {"entity": label, "score": probabilities[idx][pred], "word": tokens[idx]}
        )

print(results)
```

The output will look like this:

```
[
  {'entity': 'I-PER', 'score': 0.9993828, 'word': 'S'},
  {'entity': 'I-PER', 'score': 0.99815476, 'word': '##yl'},
  {'entity': 'I-PER', 'score': 0.99590725, 'word': '##va'},
  {'entity': 'I-PER', 'score': 0.9992327, 'word': '##in'},
  {'entity': 'I-ORG', 'score': 0.97389334, 'word': 'Hu'},
  {'entity': 'I-ORG', 'score': 0.976115, 'word': '##gging'},
  {'entity': 'I-ORG', 'score': 0.98879766, 'word': 'Face'},
  {'entity': 'I-LOC', 'score': 0.99321055, 'word': 'Brooklyn'}
]
```

Here we:

- looked at the predictions for all tokens
- kept only the ones whose label was not 0
- stored their entity label, score, and token

Step 7: Use offset mapping to find start and end positions

One thing was missing from the previous result:

- where exactly each token appeared in the original sentence

For that, we use `return_offsets_mapping=True`.

```
inputs_with_offsets = tokenizer(example, return_offsets_mapping=True)
inputs_with_offsets["offset_mapping"]
```

Output:

```
[(0, 0), (0, 2), (3, 7), (8, 10), (11, 12), (12, 14), (14, 16), (16, 18), (19, 22), (23, 24), (25, 29), (30, 32),
 (33, 35), (35, 40), (41, 45), (46, 48), (49, 57), (57, 58), (0, 0)]
```

Each tuple shows the character range in the original text that corresponds to a token.

For example:

- `(12, 14)` means from index 12 to 14 in the text

`(0, 0)` is for special tokens such as `[CLS]` and `[SEP]`.

Example:

```
example[12:14]
```

Output:

```
yl
```

That means the token `##y1` matches the substring `y1` in the original text.

Step 8: Add start and end positions to the final result

```

results = []
inputs_with_offsets = tokenizer(example, return_offsets_mapping=True)
tokens = inputs_with_offsets.tokens()
offsets = inputs_with_offsets["offset_mapping"]

for idx, pred in enumerate(predictions):
    label = model.config.id2label[pred]
    if label != "O":
        start, end = offsets[idx]
        results.append(
            {
                "entity": label,
                "score": probabilities[idx][pred],
                "word": tokens[idx],
                "start": start,
                "end": end,
            }
        )

print(results)

```

Now the output will be:

```

[
  {'entity': 'I-PER', 'score': 0.9993828, 'word': 'S', 'start': 11, 'end': 12},
  {'entity': 'I-PER', 'score': 0.99815476, 'word': '##yl', 'start': 12, 'end': 14},
  {'entity': 'I-PER', 'score': 0.99590725, 'word': '##va', 'start': 14, 'end': 16},
  {'entity': 'I-PER', 'score': 0.9992327, 'word': '##in', 'start': 16, 'end': 18},
  {'entity': 'I-ORG', 'score': 0.97389334, 'word': 'Hu', 'start': 33, 'end': 35},
  {'entity': 'I-ORG', 'score': 0.976115, 'word': '##gging', 'start': 35, 'end': 40},
  {'entity': 'I-ORG', 'score': 0.98879766, 'word': 'Face', 'start': 41, 'end': 45},
  {'entity': 'I-LOC', 'score': 0.99321055, 'word': 'Brooklyn', 'start': 49, 'end': 57}
]

```

This is the same kind of result as the first pipeline output.